這章開始，我們不再只練習零散程式碼，而是要整合前面學到的技術，打造一套完整的「新聞輿情抓取系統」。

這類系統常用於品牌監控、技術趨勢分析、產品評價追蹤等情境，核心概念就是：透過關鍵字，自動蒐集與整理網路上的資訊。

A-1 專題規劃：設定目標網站與關鍵字

一、觀念介紹

- 什麼是輿情監控？
  - 根據特定關鍵字，自動搜尋新聞、論壇或評論資料。
  - 重點在於掌握來源、時間與內容。
- 目標網站的選擇原則
  1. 穩定性：網站結構不要經常改版。
  2. 開放性：避免過度依賴登入。
  3. 結構化：HTML 標籤要有規律。
- 關鍵字的佈局
  - 主關鍵字：例如「台積電」、「生成式 AI」。
  - 排除字：過濾掉不相關內容。
  - 組合查詢：例如「Python + 爬蟲」。

二、程式範例

範例 1. 動態生成關鍵字查詢網址 (URL Encoding)

```
import urllib.parse

def generate_search_url(base_url, keyword):
    # 使用 quote 將中文字轉換為 URL 編碼 (例如: %E6%96%B0%E8%81%9E)
    encoded_keyword = urllib.parse.quote(keyword)
    return f"{base_url}/search?q={encoded_keyword}"

# 實測
yahoo_base = "https://tw.news.yahoo.com"
my_keyword = "3D 列印"
print(f"生成的網址：{generate_search_url(yahoo_base, my_keyword)}")
```

範例 2. 多關鍵字批次請求規劃

```
import requests
import time

# 設定要觀測的關鍵字清單
monitoring_list = ["台股行情", "半導體產業", "永續發展"]

for kw in monitoring_list:
    print(f"🔍 正在初始化【{kw}】的新聞抓取任務...")
    # 這裡先模擬發送請求，下一節會加入實質爬取邏輯
    time.sleep(1) # 模擬網路延遲
    print(f"✅ {kw} 任務準備就緒。")
```

範例 3. 初步分析：過濾不相關的內容

```
def check_relevance(title, content, keyword):
    """檢查標題或內容中是否真的包含關鍵字"""
    if keyword in title or keyword in content:
        return True
    return False

# 測試
test_title = "最新科技趨勢：積層製造技術解析"
test_keyword = "3D 列印"

if check_relevance(test_title, "", test_keyword):
    print("符合需求，存入資料庫。")
else:
    print("相關度過低，略過。")
```

三、程式說明

1. urllib.parse.quote：
2. 能將中文轉成網址可接受的編碼格式，避免網址錯誤。
3. 批次處理邏輯：
4. 將關鍵字放進 List 後用 for 迴圈處理，方便未來擴充。
5. 資料預處理：
6. 先過濾不相關資料，再存入資料庫，可減少雜訊。

四、常見錯誤

- 關鍵字過於廣泛：

像「新聞」這類詞會抓到大量無關資料。

- 忽略 User-Agent：

很多網站會封鎖 requests 的預設身分。

- 硬寫死網址：

把關鍵字直接寫進網址，後續維護會很麻煩。

五、延伸重要觀念

• Robots.txt 尊重原則：

抓取前先確認網站是否允許爬蟲。

- 資料時效性 (Recency)：

輿情系統通常只關心最近幾天的內容。

- 專案結構化：

本節先規劃搜尋目標，下一節會開始處理自動翻頁。

A-2 自動分頁抓取邏輯開發

學會建立搜尋網址後，下一步就是讓爬蟲能自動翻頁，持續抓取更多資料。

大部分網站的搜尋結果都有分頁，因此必須觀察網址的變化規律。

一、觀念介紹

- 分頁的兩大主流技術：
  1. URL 參數型：
  2. 網址中有 page=2 或 start=20 等規律。
  3. 按鈕觸發型：
  4. 網址不變，需要點擊「載入更多」或無限滾動。
- 迴圈控制策略：
  - for 迴圈：適合抓固定頁數。
  - while 迴圈：適合抓到沒資料為止。

二、程式範例

範例 1. 基於網址規律的 for 迴圈抓取

```
import requests
import time

base_url = "https://example-news.com/search"
keyword = "AI"

# 假設我們要抓取前 3 頁
for page in range(1, 4):
    params = {
        "q": keyword,
        "page": page
    }
    print(f"正在爬取第 {page} 頁...")

    # 模擬發送請求
    # res = requests.get(base_url, params=params)
        # 這裡加入 sleep 避免請求過快被封鎖
    time.sleep(1.5)

print("任務完成。")
```

範例 2. 使用 while 迴圈直到沒有資料

```
import requests

page = 1
while True:
    print(f"檢查第 {page} 頁...")
    # 這裡假設一個模擬的回傳資料
    # 實戰中應使用 res = requests.get(...).json() 或 BeautifulSoup 解析
    mock_data = ["新聞 1", "新聞 2"] if page < 5 else []

    if not mock_data:
        print("沒有更多資料了，停止爬取。")
        break

    print(f"成功抓取第 {page} 頁，共有 {len(mock_data)} 則內容。")
    page += 1
```

範例 3. 處理「偏移量 (Offset)」類型的分頁

```
import requests
keyword = "半導體"
limit = 10 # 每頁顯示 10 筆

for i in range(3):
    # 邏輯：第一頁 start=0, 第二頁 start=10, 第三頁 start=20
    start_val = i * limit
    url = f"https://news.test.com/api?q={keyword}&start={start_val}&limit={limit}"
    print(f"請求網址：{url}")
```

三、程式說明

1. range(1, 4)：
2. 會產生 1、2、3，常用於控制頁數範圍。
3. time.sleep()：
4. 避免請求速度過快，被網站判定為機器人。
5. params 字典：
6. 比手動拼接網址更整潔，也較不容易出錯。

四、常見錯誤

- 無限迴圈 (Infinite Loop)：

忘記 break 或 page += 1，程式可能永遠跑不完。

- 偏移量計算錯誤：

start 值算錯可能導致重複抓取或漏資料。

- 忽略最後一頁特徵：

有些網站最後一頁不會回傳空資料，而是顯示提示文字。

五、延伸觀念

- 設定「硬上限」(Hard Limit)：

即使使用 while，也建議設定最大頁數保護機制。

- 隨機延遲 (Random Sleep)：

使用 random.uniform(1, 3) 模擬真人操作。

- 資料去重 (De-duplication)：

可利用標題或網址避免重複存檔。

A-3 資料存檔：同步存入 CSV 與 SQLite

抓到大量新聞後，不能只顯示在終端機，而要真正保存下來。

CSV 適合用 Excel 查看，SQLite 則適合搜尋、去重與長期管理。

這節課會實作「CSV + SQLite 雙重存檔」。

一、觀念介紹

- CSV (Comma-Separated Values)：
  - 優點：格式通用，可直接用 Excel 開啟。
  - 缺點：資料量大時搜尋慢，不適合複雜查詢。
- SQLite：
  - 優點：Python 內建、免安裝伺服器，支援 SQL 查詢。
  - 關鍵功能：可設定 UNIQUE 避免重複資料。
- 同步存檔流程：
  - 爬到資料 → 先存 SQLite → 若成功再寫入 CSV。

二、程式範例

範例 1. 使用 csv.DictWriter 存檔

這是處理爬蟲資料最推薦的方式，因為抓到的資料通常就是字典格式。

```
import csv

news_data = [
    {"title": "3D 列印新趨勢", "date": "2026-04-20", "link": "https://test.com/1"},
    {"title": "AI 改變世界", "date": "2026-04-21", "link": "https://test.com/2"}
]

# utf-8-sig 可以確保 Excel 打開中文不會亂碼
with open('news_results.csv', 'a', newline='', encoding='utf-8-sig') as f:
    writer = csv.DictWriter(f, fieldnames=["title", "date", "link"])

    # 如果檔案是空的，就寫入表頭
    if f.tell( ) == 0:
        writer.writeheader( )

    writer.writerows(news_data)
```

範例 2. SQLite 資料庫初始化與寫入

建立一個名為 news.db 的檔案，並設定「連結」為唯一值。

```
import sqlite3

# 1. 連結資料庫 (不存在會自動建立)
conn = sqlite3.connect('news_project.db')
cursor = conn.cursor( )

# 2. 建立資料表 (設定 link 為 UNIQUE)
cursor.execute('''
    CREATE TABLE IF NOT EXISTS news (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT,
        date TEXT,
        link TEXT UNIQUE
    )
''')

# 3. 插入資料 (使用 IGNORE 確保重複時不崩潰)
sample_news = ("股市大漲", "2026-04-22", "https://news.com/stock-1")
cursor.execute('INSERT OR IGNORE INTO news (title, date, link) VALUES (?, ?, ?)',
sample_news)
conn.commit( )
conn.close( )
```

範例 3. 雙軌並行：實戰存檔函數

這是在專題主程式中會呼叫的完整邏輯。

```
def save_data(data_dict):
    # 存入 SQLite
    try:
        conn = sqlite3.connect('news_project.db')
        cur = conn.cursor( )
cur.execute('INSERT INTO news (title, date, link) VALUES (:title, :date, :link)', data_dict)
        conn.commit( )
```

```
            # 如果 SQLite 寫入成功 (沒重複)，才寫入 CSV 確保同步
            with open('news_archive.csv', 'a', newline='', encoding='utf-8-sig') as f:
                writer = csv.DictWriter(f, fieldnames=data_dict.keys( ))
                if f.tell( ) == 0: writer.writeheader( )
                writer.writerow(data_dict)
        except sqlite3.IntegrityError:
            print(f"跳過重複資料：{data_dict['title']}")
        finally:
            conn.close( )
```

三、程式說明

1. utf-8-sig：避免 Excel 開啟中文 CSV 時出現亂碼。
2. INSERT OR IGNORE：
3. 當新聞已存在時，自動跳過，避免重複存入。
4. 參數化查詢 (? 或 :key)：
5. 避免 SQL Injection，也能防止單引號造成 SQL 錯誤。

四、常見錯誤

- 忘記 conn.commit( )：

沒 commit，資料不會真正寫入資料庫。

- CSV 被 Excel 開啟：

Excel 佔用檔案時，Python 會出現 PermissionError。

- 重複寫入表頭：

若每次都 writeheader( )，CSV 中會一直出現標題列。

建議使用 f.tell( ) == 0 判斷。

五、延伸重要觀念

- 資料去重 (De-duplication)：

通常使用「網址 link」當唯一鍵，比標題更可靠。

- 資料庫工具：

可使用 DB Browser for SQLite 查看 SQLite 內容。

- 大量資料的處理：

當資料很多時，CSV 寫入會變慢，通常會先存資料庫，再統一匯出 Excel。

A-4 成果展示與 Debug 經驗分享

這是專題的最後一步。

現在要把抓取、翻頁、存檔整合成完整系統，並學會處理真實世界中的錯誤。

真正的工程能力，不只是能跑，而是能長時間穩定運行。

一、專題總結：輿情系統集成

一個完整的爬蟲專案通常包含以下流程。我們將前三節的邏輯打包成一個主程式：

```
import requests
from bs4 import BeautifulSoup
import time
import random
import sqlite3
import csv

# 1. 之前定義的清理與存檔函數 (簡略版)
def save_to_all(data):
    # 此處整合 A-3 的 SQLite 與 CSV 存檔邏輯
    pass

# 2. 主爬取邏輯
def run_spider(keyword, max_pages=3):
    print(f"🚀 啟動輿情監控系統：目標關鍵字 [{keyword}]")

    for page in range(1, max_pages + 1):
        try:
            # 模擬請求 (記得加入 User-Agent)
            headers = {'User-Agent': 'Mozilla/5.0...'}
            url = f"https://news.example.com/search?q={keyword}&p={page}"

            res = requests.get(url, headers=headers, timeout=10)
            res.raise_for_status() # 若狀態碼不是 200 會拋出異常

            # BeautifulSoup 解析
            soup = BeautifulSoup(res.text, 'html.parser')
            articles = soup.select('.news-item') # 假設的選取器
```

```
            if not articles:
                print("已無更多資料。")
                break

            for art in articles:
                item = {
                    'title': art.select_one('.title').text.strip( ),
                    'link': art.select_one('a')['href'],
                    'date': time.strftime("%Y-%m-%d")
                }
                save_to_all(item)

            print(f"☑ 第 {page} 頁處理完成")
            time.sleep(random.uniform(2, 5)) # 隨機延遲是良心

        except Exception as e:
            print(f"✖ 第 {page} 頁發生錯誤: {e}")
            continue

if __name__ == "__main__":
    run_spider("3D 列印技術", max_pages=5)
```

二、職業級 Debug 經驗分享

常見問題	現象	解決方案
Connection Timeout	程式卡住後噴 Timeout 錯誤	加入 timeout=10 與 try...except
HTML 結構改版	soup.select 回傳空列表	重新檢查 CSS Selector 或 XPath
被判定為機器人	回傳 403 Forbidden	增加 sleep、隨機 UA、必要時更換 IP
資料重複存入	CSV 越跑越大	使用 UNIQUE 或 Python 去重

三、成果展示：你得到了什麼？

1. news_archive.csv：
2. 可直接用 Excel 查看的新聞報表。
3. news_project.db：
4. 可用 SQL 查詢的大型新聞資料庫。

四、延伸重要觀念：爬蟲的「維護性」

- 日誌系統 (Logging)：

大型專案建議使用 logging 模組，而不是只用 print。

- 模組化設計：

將「請求」、「解析」、「存檔」拆成不同 .py 檔案，方便維護。

- 法律與道德：

抓取標題與連結通常沒問題，但若抓全文公開展示，可能涉及版權風險。